Optimaization

Outline

Introduction

Gradient-Based Optimization

Summary

References

- *Deep Learning* by Goodfellow, Bengio and Courville - Link
  - Chapter 8: Optimization for Training Deep Models
- online resources:
  - *Deep Learning Specialization (coursera)* - Link
  - *Stanford CS231n: CNN for Visual Recognition* - Link

Outline

Introduction

Gradient-Based Optimization

Summary

Optimization in deep learning

- most difficult optimization task in DL: training
  - ➢ so important and so expensive ⇒ need specialized techniques
- mainstream: stochastic gradient descent (sgd) and its variants
- more complicated methods: _not popular__
  - ➢ second-order methods
    - often too expensive to compute/store Hessian
    - memory-efficient techniques emerging
  - ➢ convex optimization
    - its importance greatly diminished
- for clarity: this lecture focuses on unregularized supervised case

Derivatives and optimization order

- derivatives
  - first derivative (= gradient) ⇒ slope

    (Jacobian)
  - second derivative ⇒ curvature

    (Hessian)

- optimization
  - first-order algorithms

    - use only gradient (*e.g.* gradient descent)
  - second-order algorithms

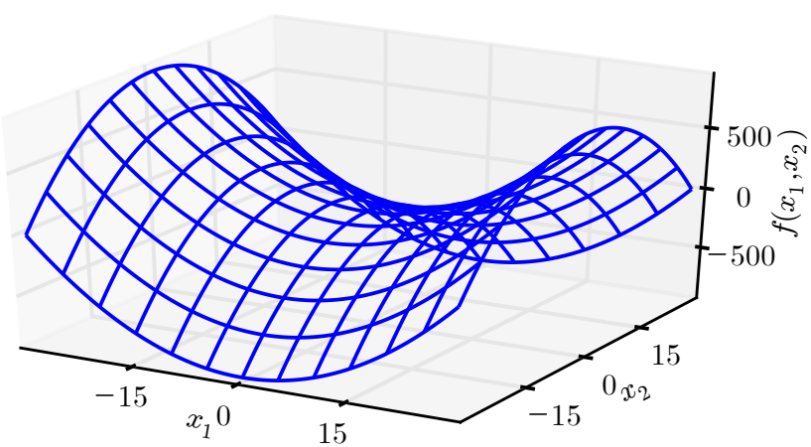
    - also use Hessian matrix (*e.g.* Newton's method)

Critical points (= stationary points)

- points with zero slope: $\nabla_x f(x) = 0$
  - derivative gives no info about which direction to move
  - three types: maxima (– curvature), minima (+ curvature), saddle points

- a saddle point: contains _both_ positive and negative curvature

$$f(\boldsymbol{x}) = x_1^2 - x_2^2$$



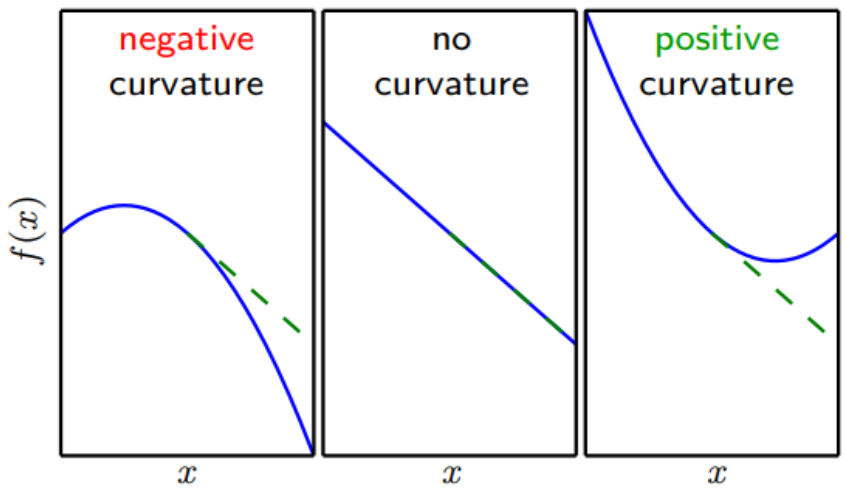
- along $x1$ axis: f curves upward
  - direction of eigenvec($\boldsymbol{H}$) with $\lambda > 0$
  - local minimum

- along $x2$ axis: f curves downward
  - direction of eigenvec($\boldsymbol{H}$) with $\lambda < 0$
  - local maximum

image source: I. Goodfellow, Y. Bengio, and A. Courville, Deep learning. MIT Press, 2016

Use of second derivative

1. to characterize critical points
2. to measure curvature
3. to predict performance of an update in _gradient_-based optimization



- negative curvature
  - f decreases faster than gradient predicts
- no curvature
  - gradient predicts the decrease correctly
- positive curvature
  - f decreases slower than gradient predicts (eventually increases)

image source: I. Goodfellow, Y. Bengio, and A. Courville, Deep learning. MIT Press, 2016

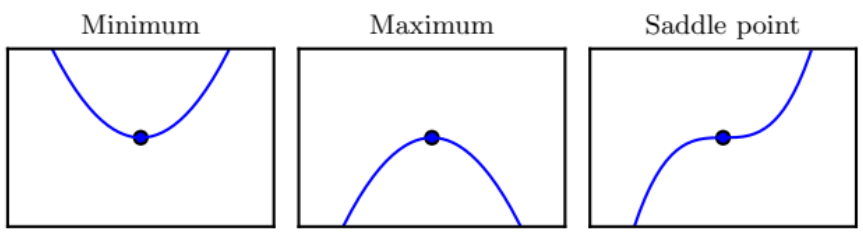
In deep learning

- our objective function has
  - ➢ many local minima + many saddle points surrounded by very flat regions

  ⇒ makes optimization very difficult (especially in high-dim space)

- we therefore usually settle for finding a very low value of f
  - ➢ not necessarily minimal in any formal sense

- recent research (Dauphin, 2014) reports
  - ➢ in high dim: _saddle points_ are much more common than local minima



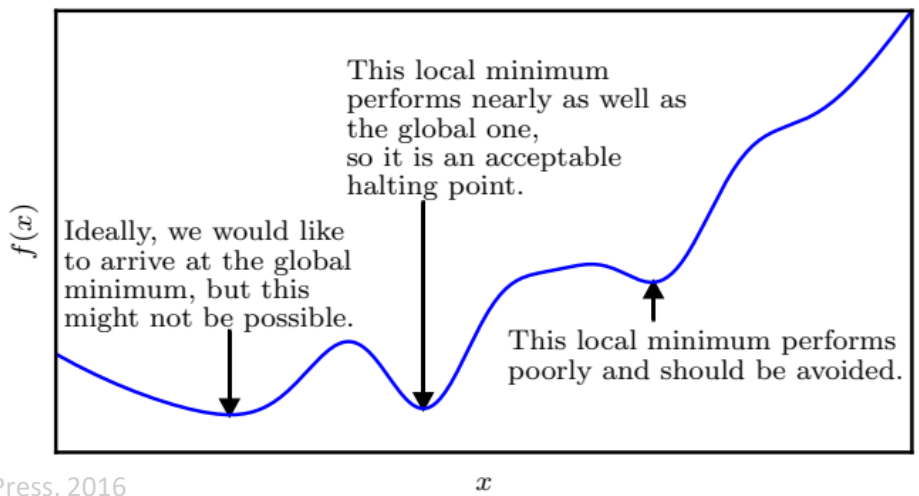


image source: I. Goodfellow, Y. Bengio, and A. Courville, Deep learning. MIT Press, 2016

Outline

Introduction

Gradient-Based Optimization

Gradient Descent and its Limitations

Exponentially Weighted Average

Gradient Descent with Momentum

Per-Parameter Adaptive Learning Rates

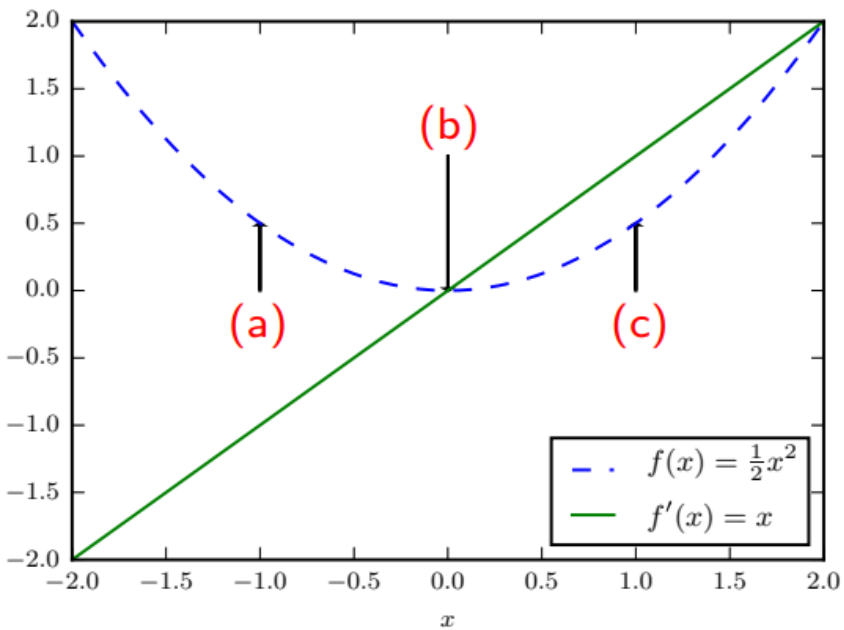
Summary

Method of gradient descent

- derivative: useful for minimizing a function
  - for small ϵ : $f\,(x - \epsilon \cdot sign\,[f'(x)]) < f(x)$

- we can thus reduce $f(x)$ by
  - moving x in small steps with opposite sign of derivative

  = method of gradient descent



$$\underbrace{\boldsymbol{x}'}_{\text{new}} = \underbrace{\boldsymbol{x}}_{\text{old}} - \underbrace{\epsilon}_{\text{learning rate}} \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$$

(a) $x < 0$: $f'(x) < 0$
$\Rightarrow$ can decrease f by moving rightward

(b) $x = 0$: $f'(x) = 0$
$\Rightarrow$ gd halts here (global min)

(c) $x > 0$: $f'(x) > 0$
$\Rightarrow$ can decrease f by moving leftward

image source: I. Goodfellow, Y. Bengio, and A. Courville, Deep learning. MIT Press, 2016

Sgd and its variants

- probably the most used optimization algorithms for ML/DL
  - can obtain an $\underbrace{\text{unbiased estimate of gradient}}_{\uparrow}$

    by taking average gradient on a minibatch of m examples

Algorithm 1 gradient descent

```
initialize θ
while stopping criterion not met do
    sample m examples: X_m = {(x^(1), y^(1)), ... (x^(m), y^(m))}
    compute gradient estimate: ĝ ← (1/m) ∇_θ Σ_{i=1}^{m} L(y^(i), ŷ^(i))   ▷ m forward props
    apply update: θ ← θ − ε ĝ                                              ▷ ε: learning rate
end while
```

- three variants (N: total number of examples)
  - $m = 1$: stochastic gradient descent (sgd)
  - $1 < m < N$: minibatch sgd (typical m: 64, 128, 256, 512)
  - $m = N$: batch gradient descent

Properties of sgd: good ones

- property #1:

  computation time per update does not grow with # of training examples

  - most important property of sgd/minibatch/online gradient-based optimization

    $\Rightarrow$ allows convergence even when # of training examples becomes large

- property #2 (see textbook):

  sgd works better in practice than its theoretical analysis says

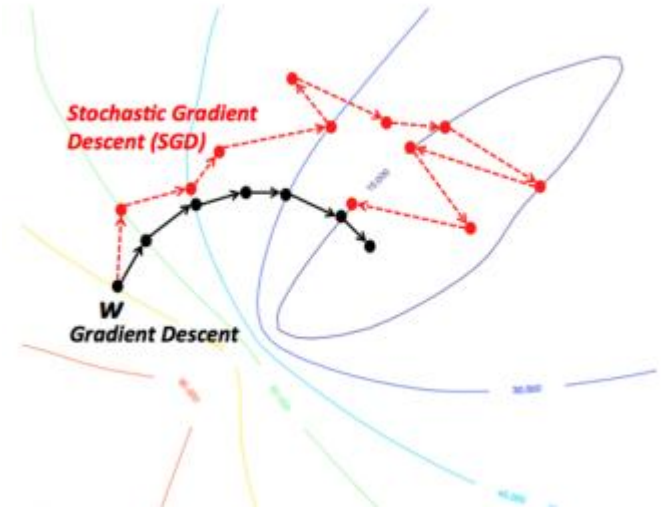
  - some benefits of sgd: obscured in asymptotic analysis

Properties of sgd: bad ones

- sgd may suffer in the following situations:
  - local minima/saddle points

zero gradient
⇒ gradient descent gets stuck

  - gradient noise



  - poor _conditioning_ of H

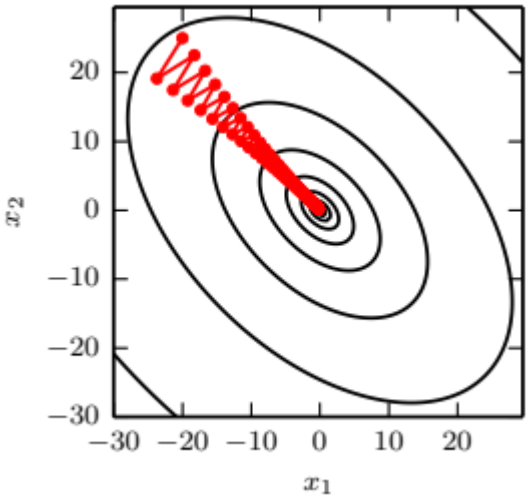


image sources: Fei-Fei Li, J. Johnson, S. Yeung, CS231n: Convolutional Neural Networks for Visual Recognition (2017), http://cs231n.stanford.edu/2017/index.html; https://wikidocs.net/3413; I. Goodfellow, Y. Bengio, and A. Courville, Deep learning. MIT Press, 2016

Ravine

- Chloe Kim (2018 Olympic Champion, Women's Snowboard Halfpipe) - link

image source: Sean M. Haffey (Getty Images)

Poor conditioning of $\boldsymbol{H}$

- consider a point x in multiple dimensions:
  - ➢ different second derivative for each direction

- condition number of Hessian $\boldsymbol{H}$ at x
  - ➢ measures how much the _second_ derivatives differ from each other
  - ➢ recall: condition number of a matrix with eigenvalues { λ }:

$$\max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right|$$

- when $\boldsymbol{H}$ has a large condition number ("poorly conditioned")
  1. gradient descent performs poorly

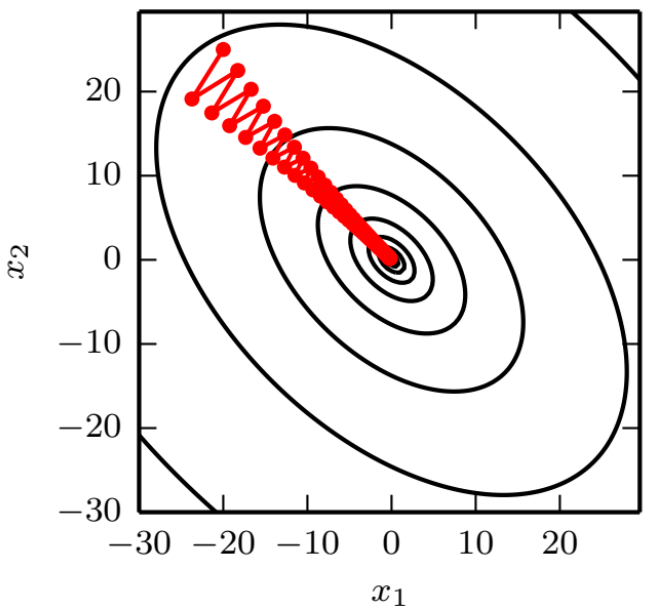
  $\because$ in a direction, derivative increases rapidly; in another, it increases slowly

  ➢ gradient descent[1] is unaware of this change in the derivative

  2. it is difficult to choose a good step size ϵ

[1] it does not know it needs to explore preferentially in the direction where derivative remains negative for longer

example:



- assume: Hessian $\boldsymbol{H}$ has condition number 5
  - ➢ most curvature: 5 times more curvature than least (a long canyon)
  - most curvature: direction [1, 1] ↗
  - least curvature: direction [1, −1] ↘

- gradient descent (red lines): slow (zig-zag)

- by contrast: methods considering $\boldsymbol{H}$
  - ➢ can predict: the steepest direction is not promising
    (large $\lambda > 0 \Rightarrow$ large positive curvature $\Rightarrow$ bad; see page 8)

- how to handle poor conditioning _without_ directly considering $\boldsymbol{H}$?

image source: I. Goodfellow, Y. Bengio, and A. Courville, Deep learning. MIT Press, 2016

Outline

Introduction

Gradient-Based Optimization

Gradient Descent and its Limitations

Exponentially Weighted Average

Gradient Descent with Momentum

Per-Parameter Adaptive Learning Rates

Summary

Exponentially weighted moving average (EWMA)

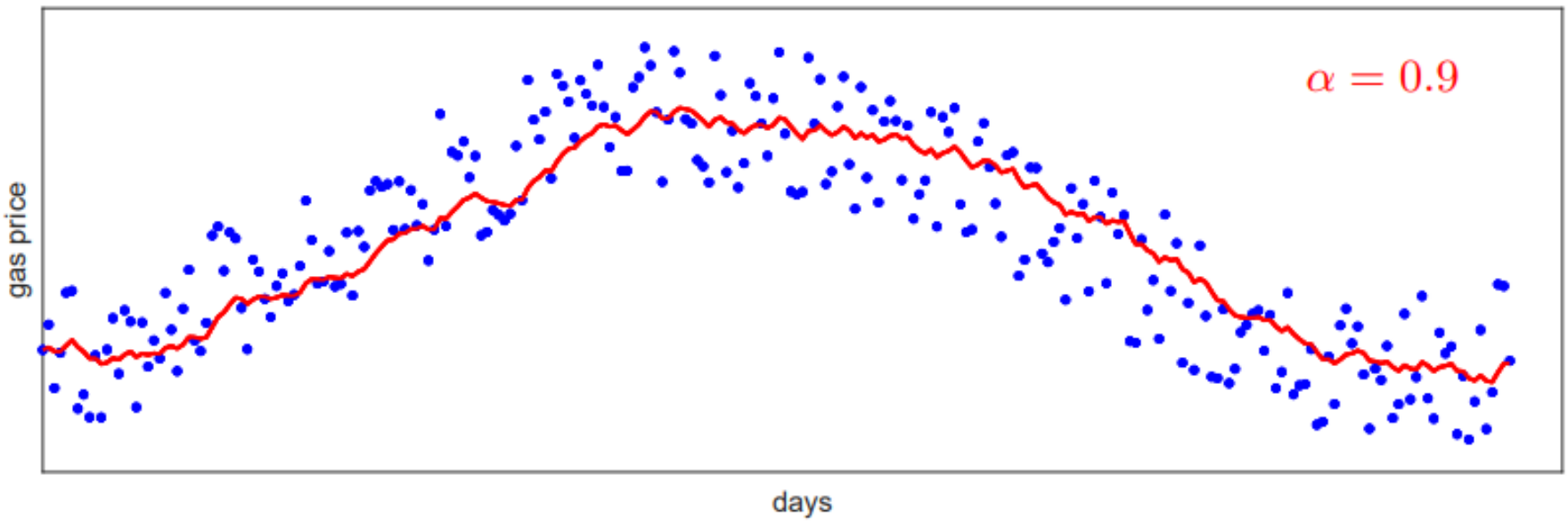
- given: time series $g_1, g_2, \ldots$
- EWMA defined as:

$$v_t = \begin{cases} g_1 & t = 1 \\ \alpha v_{t-1} + (1-\alpha) g_t & t > 1 \end{cases}$$

  - v_t: EWMA at time t
  - g_t: observation at t
  - $\alpha \in [0, 1]$: degree of weighting decrease (constant _smoothing_factor)

- example: gas price over time
  - blue dot: gas price g
  - red curve: EWMA v

$$v_t = \alpha \cdot \underbrace{v_{t-1}}_{\text{previous EWMA}} + (1-\alpha) \cdot \underbrace{g_t}_{\text{current observation}}$$



Properties of EWMA

- effective weighting decreases exponentially over time:

$$
\begin{aligned}
v_t &= \alpha v_{t-1} + (1-\alpha) g_t \\
&= \alpha \left[\alpha v_{t-2} + (1-\alpha) g_{t-1}\right] + (1-\alpha) g_t \\
&\vdots \\
&= \alpha^k v_{t-k} + (1-\alpha) \underbrace{\left[g_t + \alpha g_{t-1} + \alpha^2 g_{t-2} + \cdots + \alpha^{k-1} g_{t-k+1}\right]}_{\text{weight exponentially decreases toward the past}}
\end{aligned}
$$

⇒ thus called " _exponentially_ weighted"

- approximation[2]

$$
\begin{aligned}
v_t &= (1-\alpha)g_t + \alpha v_{t-1} \\
&= (1-\alpha)\left[g_t + \alpha g_{t-1} + \alpha^2 g_{t-2} + \alpha^3 g_{t-3} + \cdots\right] \\
&= \frac{g_t + \alpha g_{t-1} + \alpha^2 g_{t-2} + \cdots}{1 + \alpha + \alpha^2 + \cdots} \quad \Rightarrow \quad \text{weighted average formula}
\end{aligned}
$$

- in such a formula, denominator = effective number of observations:

$$1 + \alpha + \alpha^2 + \cdots = \frac{1}{1-\alpha}$$

- bottom line:

$$\boxed{v_t \approx \text{average over } \frac{1}{1-\alpha} \text{ last time points}}$$

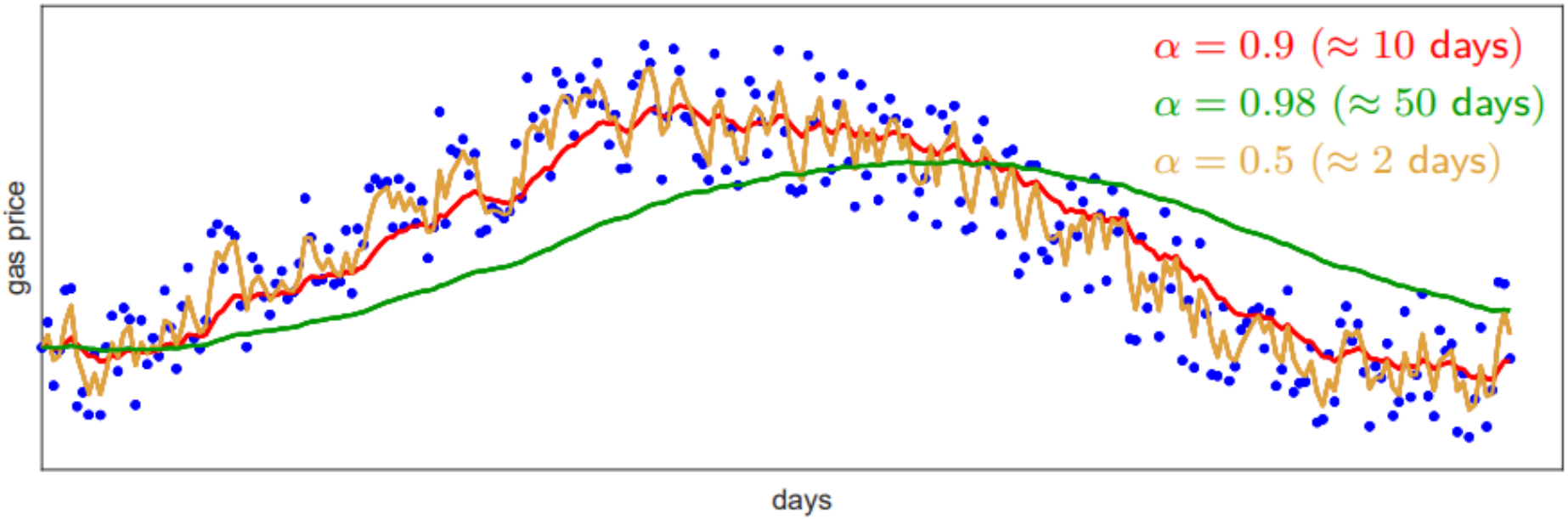
e.g. $\alpha = 0.9 \Rightarrow$ average over $1/(1-0.9) = 10$ points
$\alpha = 0.98, 0.5 \Rightarrow$ average over 50, 2 points, respectively

[2] recall: $\frac{1}{1-\alpha} = 1 + \alpha + \alpha^2 + \cdots$

Effect of smoothing factor α

- higher α (= more weight to past, less weight to present)
  - ➢ smoother curve ← averaging over more days
  - ➢ shifted further ← averaging over larger window

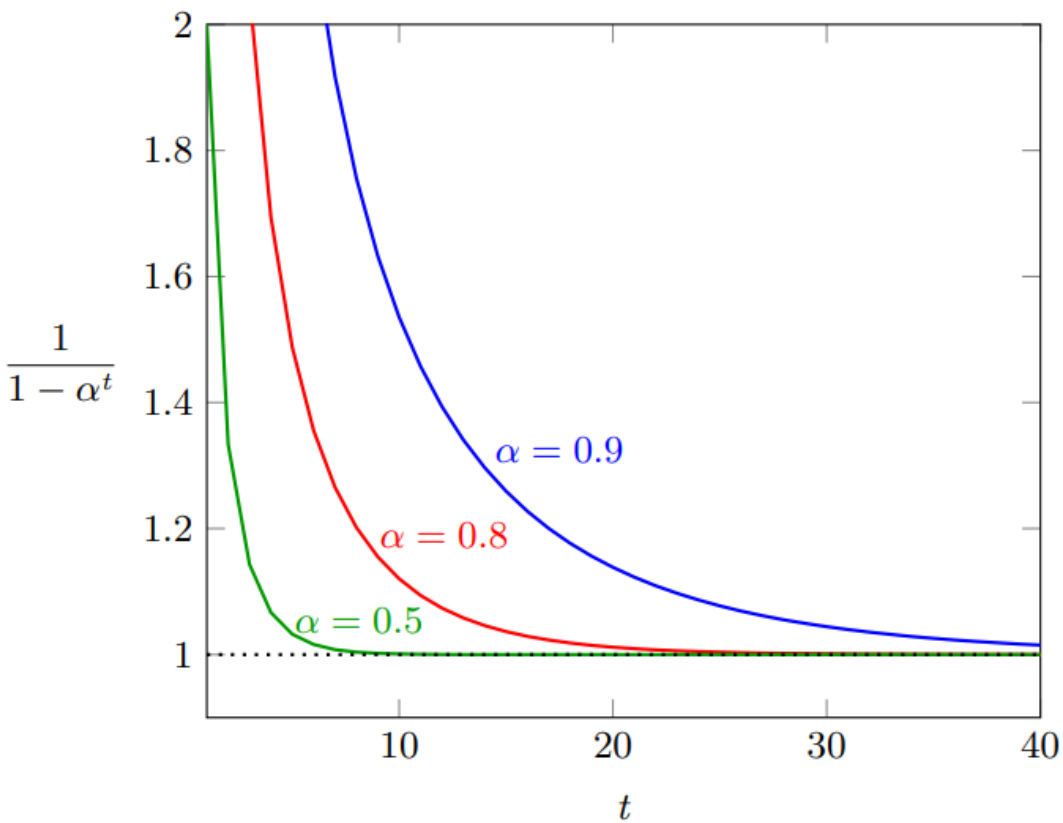
  ⇒ curve adapts more _slowly_ to changes with more latency



Bias correction

- first few iterations: inaccurate average (have not seen enough samples)
  - ➢ instead of v_t, we thus use:

$$\frac{v_t}{1-\alpha^t}$$



Outline

Introduction

Gradient-Based Optimization

Gradient Descent and its Limitations

Exponentially Weighted Average

Gradient Descent with Momentum

Per-Parameter Adaptive Learning Rates

Summary

Method of momentum

- sgd: very popular but sometimes slow

- method of momentum (Polyak, 1964):
  - ➢ designed to accelerate learning, especially in the face of
    - high curvature
    - small but consistent gradients
    - noisy gradients
  - ➢ can be combined to existing sgd variants

- common algorithm
  - ➢ accumulates exponentially decaying moving average of past_gradients_
  - ➢ then continues to move in their direction

Gradient descent with momentum

- idea: compute EWMA of gradients and use it to update weights
  - ➢ works almost always faster than standard gradient descent
- in physics
  - ➢ momentum = mass · velocity
  - ➢ for unit mass: momentum = velocity
  - ➢ smoothing factor α: friction
- sometimes
  - ➢ smoothing factor α

  ⇒ called momentum (misnomer)

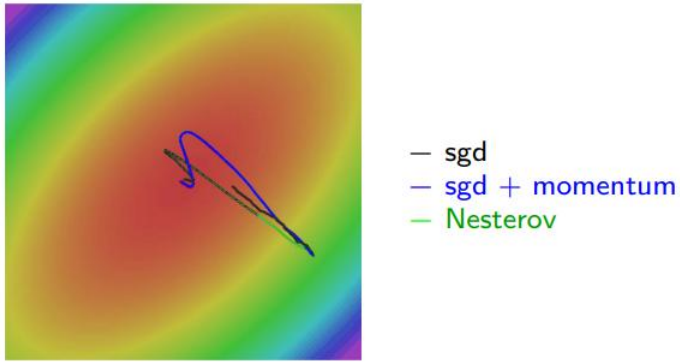


image source: Fei-Fei Li, J. Johnson, S. Yeung, CS231n: Convolutional Neural Networks for Visual Recognition (2017), http://cs231n.stanford.edu/2017/index.html

Three (equivalent) forms of sgd + momentum

- let $\boldsymbol{g} \triangleq \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ ($\boldsymbol{\theta}$ represents $\boldsymbol{W}$ and $\boldsymbol{b}$ altogether)

$\boxed{\text{form 1}}$

$$\begin{aligned} \boldsymbol{v} &\leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g} \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \boldsymbol{v} \end{aligned} \quad \Longrightarrow \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \left(\boldsymbol{g} + \frac{\alpha}{-\epsilon} \boldsymbol{v} \right)$$

$\boxed{\text{form 2}}$

$$\begin{aligned} \boldsymbol{v} &\leftarrow \alpha \boldsymbol{v} + (1-\alpha) \boldsymbol{g} \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \epsilon \boldsymbol{v} \end{aligned} \quad \Longrightarrow^{3} \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \tilde{\epsilon} \left(\boldsymbol{g} + \frac{\alpha}{1-\alpha} \boldsymbol{v} \right)$$

$\boxed{\text{form 3}}$

$$\begin{aligned} \boldsymbol{v} &\leftarrow \alpha \boldsymbol{v} + \boldsymbol{g} \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \epsilon \boldsymbol{v} \end{aligned} \quad \Longrightarrow \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \left(\boldsymbol{g} + \alpha \boldsymbol{v} \right)$$

- bottom line: $\boldsymbol{\theta}$ is updated by linear _combination_ of gradient and velocity

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \left(\underbrace{\boldsymbol{g}}_{\text{gradient}} + \text{constant} \cdot \underbrace{\boldsymbol{v}}_{\text{velocity}} \right)$$

[3] $\tilde{\epsilon} \triangleq \epsilon(1 - \alpha)$

Nesterov momentum

- difference from standard momentum:
  - _where_ gradient $g = \nabla_\theta J$ is evaluated

standard momentum	Nesterov momentum
g evaluated at current position $\boldsymbol{\theta}$ (red circle)	g evaluated at "lookahead" position $\boldsymbol{\theta} + \alpha \boldsymbol{v}$ (green circle)

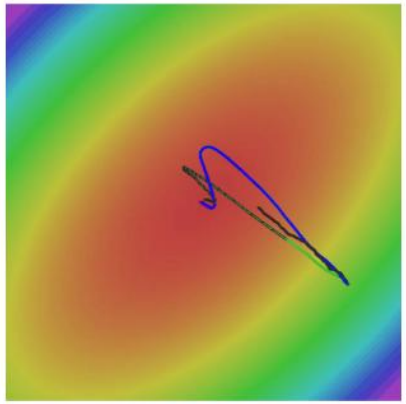
- rationale: momentum is about to carry us to a new position
  - make sense to evaluate g at new position instead of "old/stale" position

image source: Fei-Fei Li, J. Johnson, S. Yeung, CS231n: Convolutional Neural Networks for Visual Recognition (2017), http://cs231n.stanford.edu/2017/index.html

- Nesterov momentum update rule:

$$v \leftarrow \alpha v - \epsilon \nabla_\theta J(\theta + \alpha v)$$

$$\theta \leftarrow \theta + v$$



— sgd
— sgd + momentum
— Nesterov

- Nesterov momentum
  - ➢ gradient is evaluated after the current velocity is applied

  ⇒ interpreted as adding a correction factor to standard momentum

- advantages
  - ➢ stronger theoretical converge guarantees for convex functions
  - ➢ consistently works slightly better than standard momentum

image source: Fei-Fei Li, J. Johnson, S. Yeung, CS231n: Convolutional Neural Networks for Visual Recognition (2017), http://cs231n.stanford.edu/2017/index.html

Outline

Introduction

Gradient-Based Optimization

Gradient Descent and its Limitations

Exponentially Weighted Average

Gradient Descent with Momentum

Per-Parameter Adaptive Learning Rates

Summary

Per-parameter adaptive learning rates

- optimization methods explained so far
  - ➢ set learning rate ϵ globally and equally for all parameters

- methods presented now: AdaGrad, RMSProp, Adam
  - ➢ adaptively tune ϵ for _each_ parameter

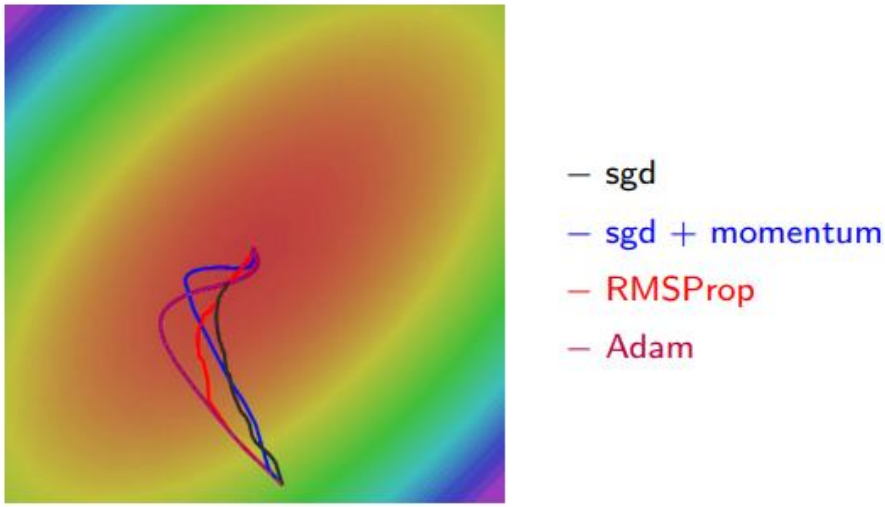
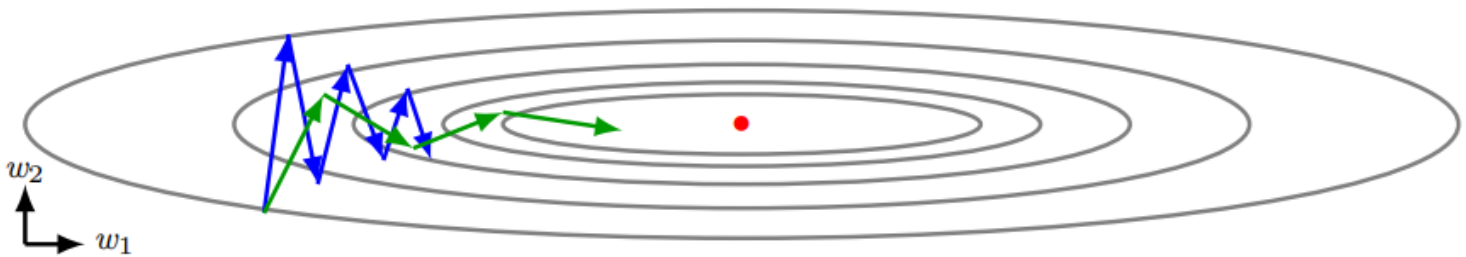


image source: Fei-Fei Li, J. Johnson, S. Yeung, CS231n: Convolutional Neural Networks for Visual Recognition (2017), http://cs231n.stanford.edu/2017/index.html

Motivation

- recall: limitation of gradient descent



  - goal: move horizontally
  - problem: huge vertical oscillations

- solution: we want to
  - slow down learning vertically
  - speed up (or at least not slow down) learning horizontally

- how to implement this idea (without relying on $\boldsymbol{H}$ explicitly)?

image modified from: Ng, Deep Learning (Coursera), https://www.coursera.org/specializations/deep-learning

AdaGrad

- individually adapts learning rate of each direction (*i.e.* each parameter)
  - steep direction (large $\frac{\partial J}{\partial \theta_j}$): slow down learning
  - gently sloped direction (small $\frac{\partial J}{\partial \theta_j}$): speed up learning

- adjusts learning rates per parameter

$$\epsilon_j = \frac{\epsilon}{\sqrt{\sum_{\text{all previous iterations}} (g_j \cdot g_j)}}$$

  - ϵ : global learning rate
  - ϵ_j : learning rate of dimension j (parameter θ_j)
  - $g_j = \frac{\partial J(\theta)}{\partial \theta_j}$: gradient wrt dimension j

- net effect:
  - greater progress in more _gently_ sloped directions

- downside (esp in deep learning)
  - ➢ monotonically decreasing : too aggressive
    - ⇒ stops learning too early

- TF: AdagradOptimizer
  - ➢ but do not use it for neural nets

- Adadelta: an extension of Adagrad
  - ➢ restricts the window of accumulated past gradients to some fixed size
    - ⇒ reduces aggressive, monotonically decreasing learning rate

RMSProp (root-mean-square prop)

- modifies AdaGrad to perform better in non-convex setting
  - ➢ changes gradient accumulation to EWMA

- use of exponentially decaying average allows RMSprop to
  - ➢ discard history from _extreme_ past
  - ⇒ converge rapidly after finding a convex bowl

- comparison ($\boldsymbol{r}$: accumulation variable)

AdaGrad	RMSprop
$\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$	$\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1-\rho)\boldsymbol{g} \odot \boldsymbol{g}$
$\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\sqrt{\delta + r}} \odot \boldsymbol{g}$	$\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\sqrt{\delta + r}} \odot \boldsymbol{g}$
$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$

  - ➢ decay rate ρ: hyperparameter (typically 0.9, 0.99, 0.999)

Adam (adaptive moment estimation)

- idea: RMSProp + momentum
  - with bias correction

- for each iteration:
  1. compute gradient g
  2. update first moment: $s \leftarrow \rho_1 s + (1 - \rho 1) g$ ← "momentum"
  3. update second moment: $r \leftarrow \rho_2 r + (1 - \rho_2) g \odot g$ ← "RMSProp"
  4. bias correction:

$$\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}, \qquad \hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$$

  5. update parameter:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}} + \delta}}$$

Algorithm 2 Adam optimizer

Require:

- step size ϵ
- exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$
- small constant δ used for numerical stabilization
- initial parameters $\boldsymbol{\theta}$

initialize 1st and 2nd moment variables $\boldsymbol{s} = \mathbf{0}$, $\boldsymbol{r} = \mathbf{0}$
initialize time step $t = 0$
while stopping criterion not met **do**
    sample a minibatch $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$
    compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    $t \leftarrow t + 1$
    update biased first moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1) \boldsymbol{g}$
    update biased second moment estimate: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2) \boldsymbol{g} \odot \boldsymbol{g}$
    correct bias in first moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}$
    correct bias in second moment: $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$
    compute update: $\Delta \boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (operations applied element-wise)
    apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$
end while

- recommended values in the paper
  - learning rate ϵ : needs tuning (suggested default: 0.001)
  - for momentum ρ_1 : 0.9
  - for RMSProp ρ_2 : 0.999
  - for stability δ: 10^{-8}

- Adam: often works better than RMSProp
  - recommended as the default algorithm to use
  - alternative to Adam worth trying: sgd + Nestrov momentum

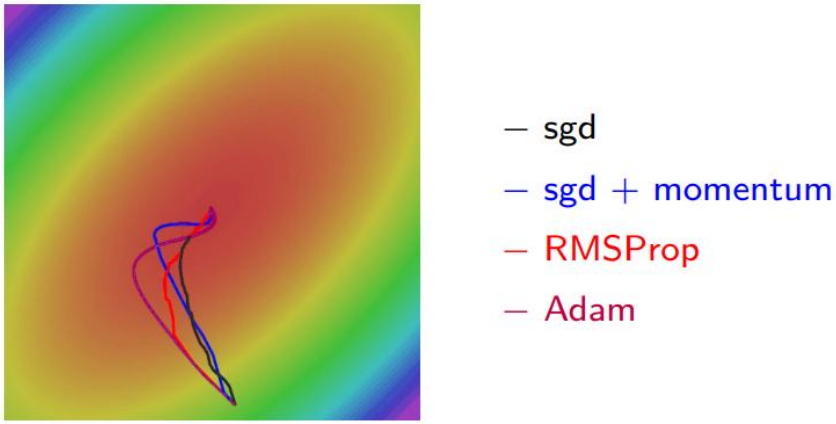
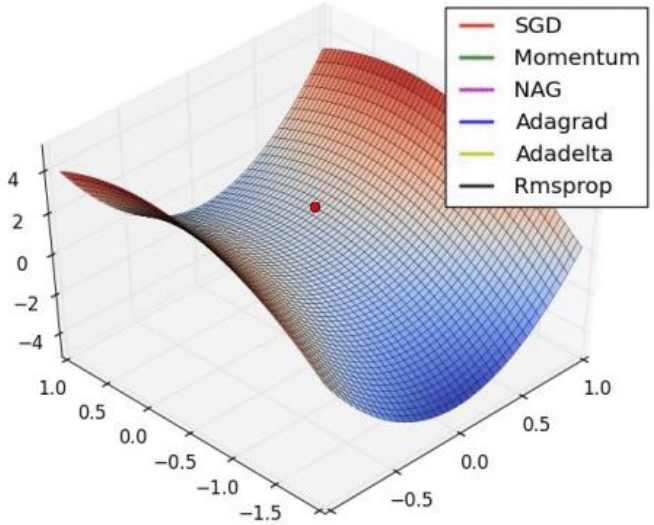
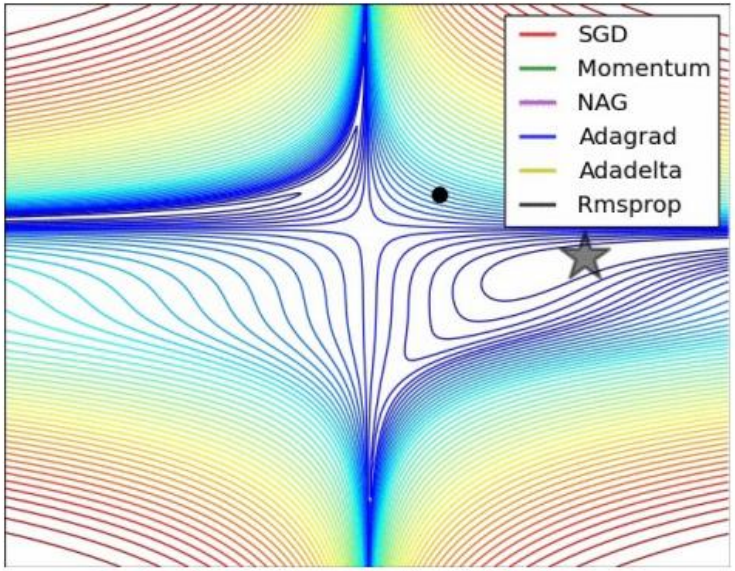


image source: Fei-Fei Li, J. Johnson, S. Yeung, CS231n: Convolutional Neural Networks for Visual Recognition (2017), http://cs231n.stanford.edu/2017/index.html

Comparison





- more information: link

image sources: http://ruder.io/optimizing-gradient-descent, http://cs231n.github.io/neural-networks-3

Outline

Introduction

Gradient-Based Optimization

Summary

Summary

- optimization in deep learning
  - ➢ mostly sgd and its variants
- gradient estimate

$$\hat{\boldsymbol{g}} = \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$$

- stochastic gradient descent (sgd)

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon_k \hat{\boldsymbol{g}}$$

- method of momentum ($\alpha \in [0, 1)$)

$$\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \hat{\boldsymbol{g}}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$$

- Nesterov momentum (corrected momentum)

$$\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left[\frac{1}{m} \sum_{i=1}^{m} L\left(\boldsymbol{f}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta} + \alpha \boldsymbol{v}), \boldsymbol{y}^{(i)} \right) \right]$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$$

- AdaGrad (r: for gradient accumulation)

$$\boldsymbol{r} \leftarrow \boldsymbol{r} + \hat{\boldsymbol{g}} \odot \hat{\boldsymbol{g}}$$
$$\Delta \boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\sqrt{\delta + r}} \odot \hat{\boldsymbol{g}}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$$

- RMSProp (gradient accumulation by EWMA)

$$\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1 - \rho) \hat{\boldsymbol{g}} \odot \hat{\boldsymbol{g}}$$
$$\Delta \boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\sqrt{\delta + r}} \odot \hat{\boldsymbol{g}}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$$

- Adam (a reasonable default choice)

$$\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1) \hat{\boldsymbol{g}} \qquad \text{(momentum)}$$
$$\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2) \hat{\boldsymbol{g}} \odot \hat{\boldsymbol{g}} \qquad \text{(RMSProp)}$$
$$\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}, \quad \hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t} \qquad \text{(bias correction)}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}} + \delta}}$$